

STRATPOINT LABS · AI LABS CASE STUDY

C1: Predicting Solar Power Output

Weather-driven output prediction, city-level solar extraction, and an LLM planning agent

Presented by Gienel Francheska D.S. Manarang

Senior AI Engineer Candidate

AGENDA

Outline

1. Problem Statement
2. General Approach
3. Assumptions & Limitations
4. Methodology
 - Data Preparation
 - Exploratory Data Analysis
 - Tools & Frameworks
 - Algorithm & Models
 - System Flow
5. Results
 - Model Performance
 - Agent Evaluation
6. Discussion & Recommendations

01 · PROBLEM STATEMENT

What needs to be solved

- Predict a city's expected solar power output from local weather patterns, using two independent, unmatched public data sources.
- Expose that prediction through an **LLM-powered natural-language agent** a solar farm developer could actually use.
- Do this end-to-end (geospatial data extraction, a trained regression model, and an LLM-driven interface) on a genuinely small, real-world-shaped dataset.

48

cities with
Global Solar Atlas (GSA) coverage

~10 yrs

daily weather
observations

5+

agent questions
demoed

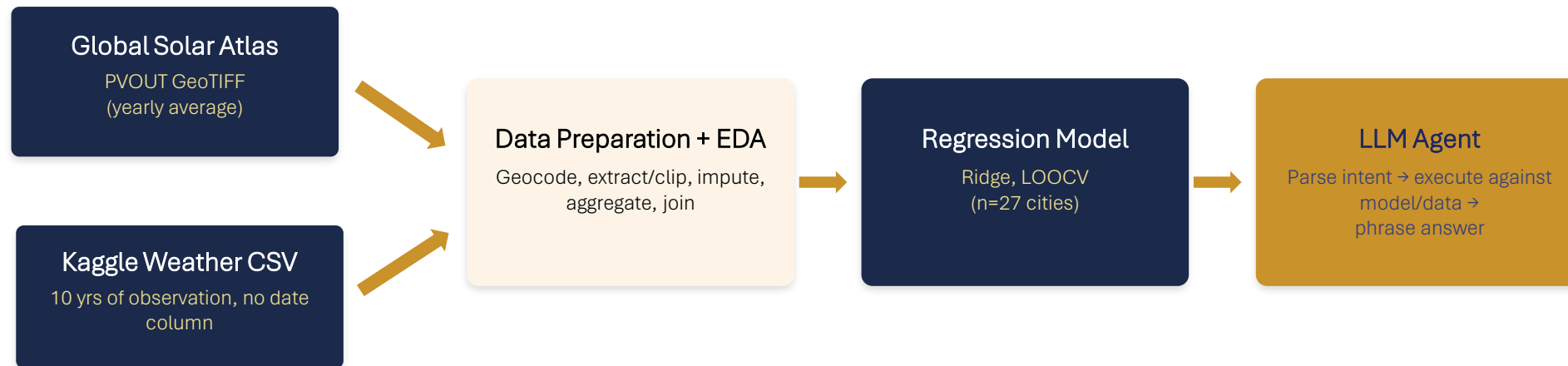
2

raw data sources
(GSA + Kaggle)

02 · GENERAL APPROACH

System diagram

Solar extraction and weather preparation converge into one modeling table feeding the regression model and LLM agent.



Natural-language questions in → grounded, numeric answers out

- Two independent pipelines (solar extraction, weather prep + EDA) converge into one modeling table.
- The agent calls the trained model / deterministic functions and only uses the LLM to parse intent and phrase the response.

03 · ASSUMPTIONS & LIMITATIONS

Key decisions

Assumptions

- PVOUT's yearly average is matched to weather stats aggregated across all available city observations. Neither source has date/monthly resolution the other can use.
- 15km fixed radius around each geocoded city point defines the area averaged for that city's solar value.
- Missing weather values imputed with each city's own mean, not the global mean to preserve city-to-city variation.
- Cities outside Global Solar Atlas's coverage (e.g. Norfolk Island) are excluded from modeling.

Limitations

- Only n=27 independent data points after EDA-driven filtering for complete weather coverage (down from 48 raw GSA-covered cities) which makes it a genuinely small-n regression problem.
- "What-if" queries (e.g. doubling rainfall) reflect a learned linear correlation, not a physically simulated causal effect.
- No separate real-world holdout set; validation is Leave-One-Out CV on the same 27 cities.
- Agent answers are only as accurate as the underlying regression; evaluation confirms the pipeline didn't corrupt values, not real-world ground truth.

From raw files to one modeling table



- **Solar extraction (rasterio):** reproject each city's buffer into the GeoTIFF's CRS, clip with rasterio.mask, average valid pixels.
- **Weather prep:** missingness checked and imputed per-city before aggregation – the reasoning behind this is detailed in the EDA on the next slide.

Why per-city imputation, and n = 27

Column missingness

Sunshine	47.5%
Evaporation	42.7%
Cloud3pm	40.2%
Cloud9am	37.8%
Pressure9am/3pm	9.8%
WindGustSpeed	6.5%
All others	< 3%

Usable cities

48

GSA-covered cities

↓ drop cities with any
100%-missing feature

27

final modeling sample

Feature selection funnel

Engineered features (mean/min/max × 16) **48**

Pass |correlation| > 0.20 vs. solar_output **33**

Final, domain-narrowed set **6**

Rainfall_mean fell below the threshold but was kept regardless – the agent's rainfall what-if intent needs it (a domain-driven exception)

- Per-city mean imputation was chosen because several cities are 100% missing certain features (global mean would silently invent data for a city that has none at all)

Tools & frameworks used



Geospatial

- rasterio
- shapely
- pyproj
- geopy



Modeling

- scikit-learn (Ridge, LassoCV)
- Comparison: Linear, Lasso, DecisionTree, RF, GB
- joblib, pandas, numpy



LLM Agent

- Ollama (local, llama3.1:8b)
- OpenAI-compatible client
- Rule-based fallback parser

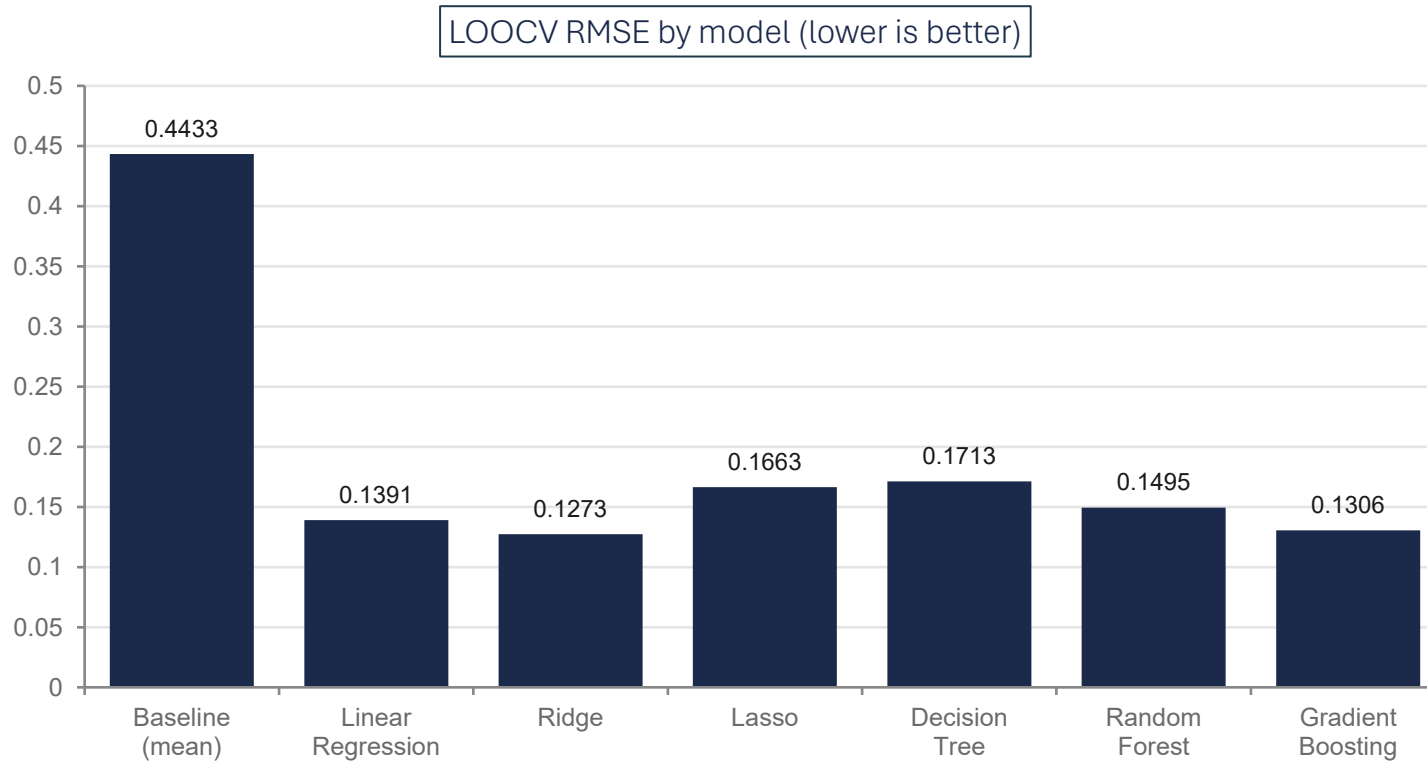


Evaluation

- Custom eval harness
- White-box unit tests
- Trajectory + value checks

No paid APIs used. The LLM runs locally (Ollama, llama3.1:8b)

Model selection under $n = 27$

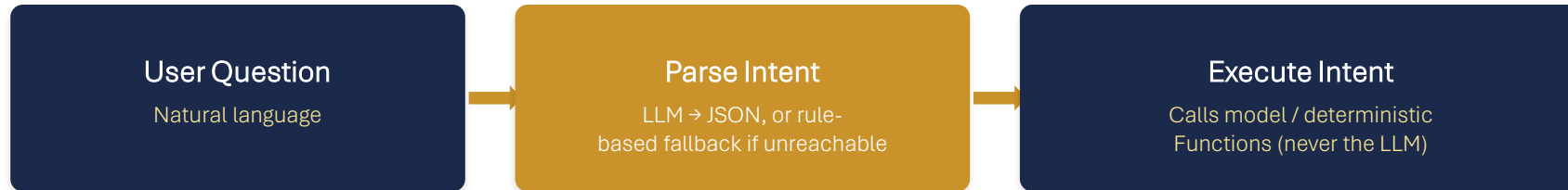


Why Ridge

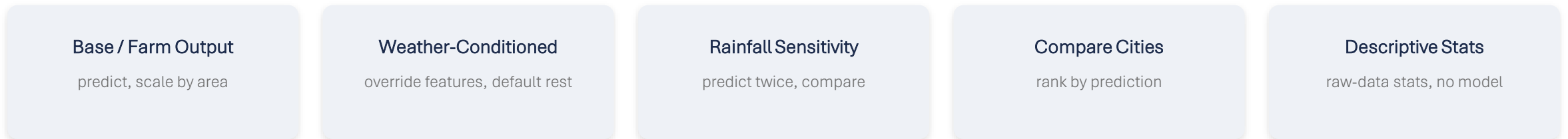
- Beats the baseline (RMSE 0.443 $\rightarrow$ 0.127) and is competitive with the best score, without tree models' overfitting risk at $n=27$.
- Regularization + a narrow, domain-informed feature set (6 variables) keeps the feature-to-sample ratio sane.
- Coefficients stay interpretable (important for the agent's rainfall what-if queries).

04 · METHODOLOGY: SYSTEM FLOW

How the agent answers a question



5 supported intents:



- Every override or default is checked against the model's observed training range and flagged if it extrapolates.
- If the LLM server is unreachable, parsing falls back to keyword rules and phrasing falls back to a template. Pipeline stays runnable end to end either way.

Intent parser prompt template

You are an intent parser for a solar-output prediction assistant.

Respond with ONLY a JSON object matching one of these shapes:

Valid city names: {city_list_str}. # normalize aliases/misspellings; never invent a city

1. {"intent": "base_or_farm_output", "city": "<city>", "hectares": <number|null>}
2. {"intent": "weather_conditioned_output", "city": "<city>", "weather": {"<feature>": <value>, ...}}
3. {"intent": "rainfall_sensitivity", "city": "<city>", "multiplier": <number, e.g. 2.0>}
4. {"intent": "compare_cities", "cities": [<city1>, <city2>, ...]}
5. {"intent": "descriptive_stats", "city": "<city>", "feature": "<feature>", "stat": "mean|median|std|...", "threshold": <number|omit>}
6. {"intent": "unknown"}

Respond with ONLY the JSON object, nothing else.

05 · RESULTS: MODEL PERFORMANCE

LOOCV results, final Ridge model

0.1273
LOOCV RMSE

0.0869
LOOCV MAE

0.9176
LOOCV R^2

Yes
Beats baseline (0.443)

Selected features (domain-informed, narrow set)

- Sunshine_mean, Cloud9am_mean, Cloud3pm_mean, MaxTemp_mean, Rainfall_mean, Humidity3pm_mean – 6 variables with a direct physical relationship to solar output.
- LassoCV run as a secondary check confirms which of these the model actually relies on. Rainfall_mean was kept deliberately despite weak standalone correlation – the agent's rainfall what-if intent needs it.

n=27 cities

05 · RESULTS: AGENT EVALUATION

3-tier evaluation harness

1 White-box

Unit tests on individual tool functions

20/20

synthetic fixtures passed (100%)

2 Trajectory

Does parsing extract the right intent + arguments?

85%

trajectory accuracy

3 Final response

Does the numeric result match direct recomputation?

80%

final-value accuracy

- Final-response checks confirm the LLM-parsed path didn't corrupt the number on the way in. BUT this is not a validation of the model's real-world accuracy (that's the LOOCV metrics from the previous slide).
- Based on [Langfuse – Example: Pydantic AI + MCP Agent Evaluation](#)

What worked, what's next

Discussion

- Ridge beating the baseline at $n=27$ (RMSE $0.443 \rightarrow 0.127$, $R^2 \approx 0.92$) is a meaningful signal that the weather-to-solar relationship is learnable at this scale – not proof it's a strong relationship in general.
- Grounding the agent's numbers entirely in the model/deterministic layer, with the LLM restricted to parsing and phrasing, was the right call for trustworthiness.
- The LLM-unreachable fallback path is a resilience feature that keeps the pipeline runnable end to end without Ollama, at the cost of reduced trajectory accuracy.

Recommendations

- Extend city coverage as new weather stations become available, to grow the sample beyond $n=27$ and reduce the LOOCV-only validation constraint.
- Add prediction intervals (not just point estimates) so the agent can communicate model uncertainty rather than implying false precision.
- Expand the eval harness's benchmark set as real usage surfaces question phrasings the current demoed questions don't cover.
- Explore monthly-resolution PVOUT if a dated weather source becomes available, addressing the temporal-alignment assumption directly.

LLM Agent Demo

STRATPOINT LABS · AI LABS CASE STUDY

C2: Extracting Data From Receipts

OCR + LLM Pipeline for Structured Information Extraction

Presented by Gienel Francheska D.S. Manarang

Senior AI Engineer Candidate

AGENDA

Outline

1. Problem Statement
2. General Approach
3. Assumptions & Limitations
4. Methodology
 - Data & Tools
 - Algorithm & Models
 - System Flow
5. Results
 - Evaluation of Extraction Strategies
 - Time Consumption
6. Discussion & Recommendations

01 · PROBLEM STATEMENT

Manual receipt processing doesn't scale

Businesses process receipts and invoices for accounting, reimbursement, and supply/demand modeling. Doing this by hand is slow, error-prone, and doesn't scale with transaction volume.

- Automatically extract the fields from the receipt image: company, date, address, total
- Handle real-world OCR noise (skewed scans, faded ink, cut-off text)
- Let a user query a receipt in plain language, not raw JSON

tan chay yee

*** COPY ***
 OJC MARKETING SDN BHD
 ROC NO: 538358-H
 NO 2 & 4, JALAN BAYU 4,
 BANDAR SERI ALAM,
 81750 MASAI, JOHOR
 Tel:07-388 2218 Fax:07-388 8218
 Email: ng@ojcgroup.com

TAX INVOICE

Invoice No : PEGIV-1030765
 Date : 15/01/2019 11:05:16 AM
 Cashier : NG CHUAN MIN
 Sales Person : FATIN
 Bill To : **THE PEAK QUARRY WORKS**
 Address : .

Description	Qty	Price	Amount
000000111	1	193.00	193.00 SR
KINGS SAFETY SHOES KWD 805			
Qty: 1	Total Exclude GST:	193.00	
	Total GST @6%:	0.00	
	Total Inclusive GST:	193.00	
	Round Amt:	0.00	
TOTAL:		193.00	
VISA CARD		193.00	
xxxxxxxxxxxx4318			
Approval Code:000			

Goods Sold Are Not Returnable & Refundable
 ****Thank You. Please Come Again.****

Figure 1. A sample receipt from the SROIE V2 dataset

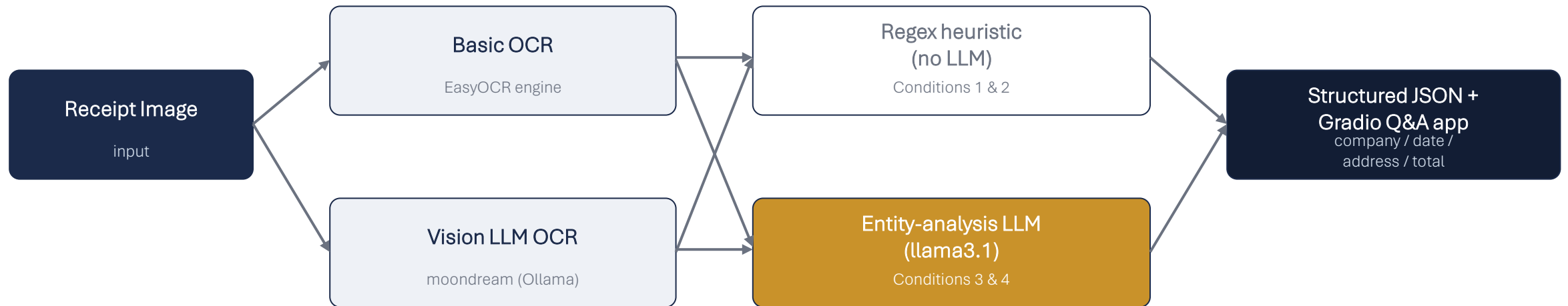
```
{
  "company": "OJC MARKETING SDN BHD",
  "date": "15/01/2019",
  "address": "NO 2 & 4, JALAN BAYU 4, BANDAR SERI ALAM, B1750 MASAI, JOHOR",
  "total": "193.00"
}
```

Figure 2. Extracted information from the receipt in Figure 1

02 · GENERAL APPROACH

Two OCR paths, one structuring stage

Every receipt runs through two independent OCR extractions, each of which can optionally be structured and corrected by an LLM – producing four comparable conditions from one pipeline.



Both OCR stages feed both the plain regex path and the LLM structuring path.

03 · ASSUMPTIONS & LIMITATIONS

Key decisions

Assumptions

- Fully local execution on CPU. No paid LLM/OCR APIs used
- SROIE V2 train + test pools combined, then sampled to N=200 receipts for a tractable, reproducible local evaluation
- Ground-truth entities normalized (case, punctuation, amount formatting) before comparison, matching how a human would judge correctness

Limitations

- moondream swapped in for llava as the vision model – smaller and faster on CPU, at some cost to transcription accuracy
- N=200 is a documented subsample, not the full dataset – results should be read as directional, not final
- Entity LLM corrects text it's given; it can't recover information the OCR stage lost entirely

Data preparation & tools

DATA PREPARATION

- 1 Combine SROIE V2 train + test splits into one pool
- 2 Sample N=200 receipts for a tractable local run
- 3 Pair each image with its ground-truth entity file
- 4 Normalize text & amounts for fair comparison

TOOLS & FRAMEWORKS

EasyOCR

Pure-Python basic OCR engine (CPU, no external binary)

Ollama

Local model runtime – no per-call API cost

moondream

Lightweight vision-language model for image OCR

llama3.1

Text LLM for entity structuring & correction

Gradio

Upload + Q&A web app

pandas

Evaluation scoring & results tables

Algorithms & models

Basic OCR

EasyOCR

CNN + CTC-based scene-text detection/recognition. Fast, deterministic, no LLM involved – the baseline for what OCR alone can do.

Vision LLM OCR

moondream

Compact vision-language model reading the receipt image directly. Chosen over llava for CPU speed – smaller footprint, faster per-receipt latency, some accuracy trade-off.

Entity structuring

llama3.1 (JSON-constrained)

Single prompt handles extraction, rectification of garbled OCR text, and completion of truncated fields – output constrained to the 4-field JSON schema.

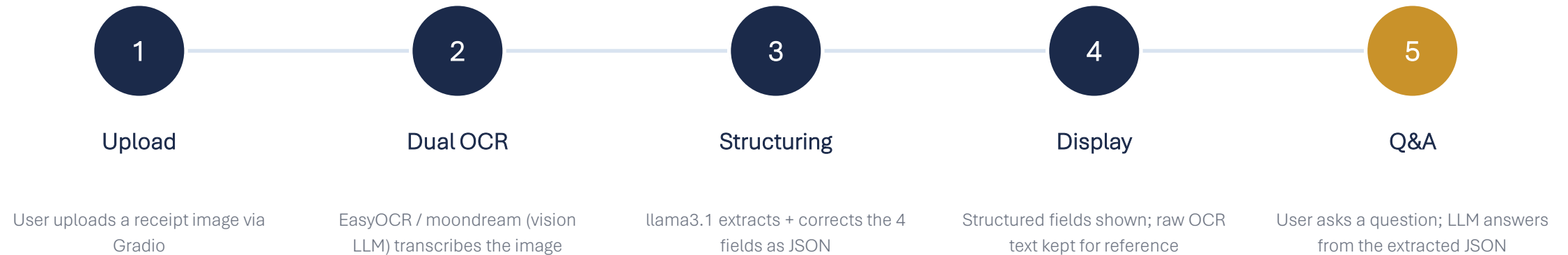
No-LLM baseline

Regex heuristics

Rule-based field guessing applied to the same raw OCR text, so 'with LLM' vs. 'without LLM' is a fair, apples-to-apples comparison.

05 · METHODOLOGY: SYSTEM FLOW

System flow: request to response



Offline evaluation reuses the same pipeline: every sampled receipt runs through all 4 conditions (not just the app's default path), and predictions are scored against SROIE ground truth to build the comparison in the next section.

05 · RESULTS: PIPELINE PERFORMANCE

4-condition comparison, n = 200 receipts

Condition	Extraction Rate	Exact Match	Avg. Similarity
Raw OCR (EasyOCR only)	83.6%	27.1%	0.465
Raw OCR + Entity LLM	99.9%	40.2%	0.842
Multimodal OCR (moondream only)	29.9%	0.4%	0.106
Multimodal OCR + Entity LLM	50.6%	1.0%	0.254

Final run: 200/200 receipts processed, 0 errors (merged train+test pool, seed=42).

Raw OCR + Entity LLM wins clearly. The LLM structuring stage lifts basic OCR from a moderate baseline to 99.9% extraction and 84.2% similarity. Both moondream-based conditions lag well behind, even with LLM structuring: moondream's transcription quality, not the entity-extraction stage, is the bottleneck. This reverses the original expectation that the multimodal path would lead.

05 · RESULTS: TIME CONSUMPTION

What it actually cost to get there

TOTAL WALL TIME: N=200 EVALUATION

Run 1	150.5 min	Crashed after 31 receipts
Run 2	82.8 min	Before memory fix – mostly fast failures
Run 3 (final)	835.3 min (~13h 55m)	154 remaining receipts, 0 errors

Total across all 3 attempts: ~1,068.6 min (~17.8h) – includes crash + OOM debugging, not just clean processing.

Steady-state, once fixed: ~300–325s/receipt (all 4 conditions combined)
 **useful number for planning future runs

PER-CONDITION TIMING (5-RECEIPT SAMPLE)

raw_ocr	44.4s	Basic OCR only
multimodal_ocr	86.0s	Vision LLM transcription only
multimodal_ocr_llm	145.2s	Vision LLM + entity LLM
raw_ocr_llm	186.0s	Basic OCR + entity LLM

Entity extraction on basic-OCR text averages 141.7s vs. just 59.1s on vision-LLM text (over 2x longer).

EasyOCR's noisier output means more corrective work for the LLM; moondream's cleaner transcription needs less fixing.

LLM post-processing cost scales with upstream OCR quality; it's not a fixed cost.

What worked, What's next

Discussion

- LLM structuring is the biggest lever. The regex baseline confirms its contribution is real.
- The regex baseline was a deliberate design choice: without it, there's no way to isolate the LLM's actual contribution.
- N=200 is large enough to see a directional pattern across conditions but should not be read as a final production benchmark.
- moondream-for-llava was a right call under the CPU-only constraint, but the accuracy cost was larger than expected; might be worth re-testing with GPU access.

Recommendations

- Run the full test set once compute allows, not just N=200
- Re-benchmark llava vs. moondream if GPU inference becomes available
- Experiment with the entity prompt (e.g., add few-shot examples to the entity prompt for harder receipt layouts)
- Add confidence flags to the app so low-confidence extractions get routed to human review
- For production, evaluate a hosted multimodal API on the accuracy-critical path

Gradio App Demo